FOUNDATION OF JAVA LAB SHEET – 6
Exception Handling — Questions Only Total
Questions: 15 | Easy/Basic: 10 | Real-World: 5

Part A — Easy / Basic Questions

Q1. Handling ArithmeticException
Write a Java program that accepts two integers from the user and divides the first number by the second number. Handle the situation where the user enters 0 as the denominator using try-catch.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter two numbers: ");
            int a = sc.nextInt();
            int b = sc.nextInt();
            System.out.println("Result: " + (a / b));
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        }
    }
}
```

OUTPUT:
Enter two numbers: 10 0
Cannot divide by zero

Concept: try, catch, ArithmeticException
Q2. Handling ArrayIndexOutOfBoundsException
Create an integer array containing 5 elements. Ask the user to enter an array index and display the corresponding element. Handle the situation where the user enters an invalid index.

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        int[] a = {10, 20, 30, 40, 50};
```

```
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter index: ");
            int i = sc.nextInt();
            System.out.println("Element: " + a[i]);
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid index");
        }
    }
}
```

OUTPUT:
Enter index: 7
Invalid index

Concept: ArrayIndexOutOfBoundsException
Q3. Handling NumberFormatException
Write a Java program that stores a number as a String and converts it into an integer using Integer.parseInt(). Handle the exception if the string does not contain a valid integer.

```
public class Main {
    public static void main(String[] args) {
        String s = "123a";

        try {
            int n = Integer.parseInt(s);
            System.out.println("Number: " + n);
        } catch (NumberFormatException e) {
            System.out.println("Invalid number");
        }
    }
}
```

OUTPUT:
Invalid number

Concept: NumberFormatException
Q4. Multiple catch Blocks Write a Java program that:
1. Accepts two numbers from the user.

2. Performs division.
3. Creates an integer array containing five elements.
4. Accepts an index from the user.
5. Displays the corresponding array element.

Handle the following exceptions using separate catch blocks:
• ArithmeticException
• ArrayIndexOutOfBoundsException
• NumberFormatException
Concept: Multiple catch blocks

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        int[] a = {10, 20, 30, 40, 50};

        try {
            System.out.print("Enter two numbers: ");
            int x = Integer.parseInt(sc.next());
            int y = Integer.parseInt(sc.next());
            System.out.println("Division: " + x / y);

            System.out.print("Enter index: ");
            int i = Integer.parseInt(sc.next());
            System.out.println("Element: " + a[i]);

        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        } catch (ArrayIndexOutOfBoundsException e) {
            System.out.println("Invalid array index");
        } catch (NumberFormatException e) {
            System.out.println("Invalid number");
        }
    }
}
```

OUTPUT:
Enter two numbers: 10 2
Division: 5
Enter index: 2
Element: 30

Q5. Using finally
Write a Java program that accepts two numbers and performs division. Use a finally block to display:
Program execution completed.
The message must be displayed whether an exception occurs or not.
Concept: finally

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            int a = sc.nextInt();
            int b = sc.nextInt();
            System.out.println("Result: " + a / b);
        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        } finally {
            System.out.println("Program execution completed.");
        }
    }
}
```

OUTPUT:

```
10
2
Result: 5
Program execution completed.
```

Q6. Using throw Write a Java program that accepts a student's marks. If the marks are: • Less than 0, or • Greater than 100 explicitly generate an exception using the throw keyword and display an appropriate error message. Concept: throw

```
import java.util.Scanner;

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        try {
            System.out.print("Enter marks: ");
            int marks = sc.nextInt();

            if (marks < 0 || marks > 100)
                throw new Exception("Invalid marks");

            System.out.println("Marks: " + marks);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

OUTPUT:

```
Enter marks: 120
Invalid marks
```

Q7. Using throws Create a method named checkAge(int age). If the age is less than 18, the method should generate an exception. Declare the exception using the throws keyword. Call the method from main() and handle the exception. Concept: throws

```
class Main {
    static void checkAge(int age) throws Exception {
        if (age < 18)
            throw new Exception("Not eligible");
        System.out.println("Eligible");
    }

    public static void main(String[] args) {
        try {
            checkAge(16);
        } catch (Exception e) {
            System.out.println(e.getMessage());
        }
    }
}
```

OUTPUT:

```
Not eligible
```

Q8. Nested try-catch Write a Java program containing a try-catch block inside another try block. • The inner try block should perform array access. • The outer try block should perform division. Handle both possible exceptions. Concept: Nested try-catch

```
public class Main {
    public static void main(String[] args) {
        try {
            int a = 10, b = 0;
            System.out.println(a / b);

            try {
                int[] x = {10, 20};
                System.out.println(x[5]);
            } catch (ArrayIndexOutOfBoundsException e) {
                System.out.println("Invalid index");
            }

        } catch (ArithmeticException e) {
            System.out.println("Cannot divide by zero");
        }
    }
}
```

OUTPUT:
Cannot divide by zero

Q9. Exception Propagation
Create three methods:
method1()
method2()
method3()
method3() should generate an ArithmeticException.
Allow the exception to propagate from method3() through method2() and method1() and finally handle it in main().

```
class Main {
    static void method3() {
        int x = 10 / 0;
    }
```

```
    static void method2() {
        method3();
    }

    static void method1() {
        method2();
    }

    public static void main(String[] args) {
        try {
            method1();
        } catch (ArithmeticException e) {
            System.out.println("Exception handled in main");
        }
    }
}
```

OUTPUT:
Exception handled in main

Q10. User-Defined Exception
Create a custom exception named InvalidMarksException. Accept marks from the user. If the marks are outside the range 0–100, throw the custom exception and display an appropriate error message. Concept: Custom exception, extends Exception, throw

```
import java.util.Scanner;

class InvalidMarksException extends Exception {
    InvalidMarksException(String msg) {
        super(msg);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter marks: ");
            int marks = sc.nextInt();
```

```
            if (marks < 0 || marks > 100)
                throw new InvalidMarksException("Invalid Marks!");

            System.out.println("Marks: " + marks);
        } catch (InvalidMarksException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

OUTPUT:
Enter marks: 120
Invalid Marks!

Q11. Bank Withdrawal System
Create a Java program for a simple bank withdrawal system.
Accept:
• Account balance
• Withdrawal amount
Apply the following rules:
1. Withdrawal amount cannot be negative.
2. Withdrawal amount cannot be greater than the available balance.
3. If the withdrawal amount is greater than the balance, create and throw a custom exception named InsufficientBalanceException.
4. Handle invalid numeric input.
5. Use a finally block to display:
Bank transaction completed.
Concepts: Custom exception, throw, multiple catch, finally, NumberFormatException

```
import java.util.Scanner;

class InsufficientBalanceException extends Exception {
    InsufficientBalanceException(String msg) {
        super(msg);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
```

```
        try {
            System.out.print("Enter balance: ");
            double balance = Double.parseDouble(sc.next());

            System.out.print("Enter withdrawal amount: ");
            double amount = Double.parseDouble(sc.next());

            if (amount < 0)
                throw new IllegalArgumentException("Withdrawal cannot be negative");

            if (amount > balance)
                throw new InsufficientBalanceException("Insufficient Balance");

            System.out.println("Withdrawal Successful");
            System.out.println("Remaining Balance: " + (balance - amount));

        } catch (NumberFormatException e) {
            System.out.println("Invalid numeric input");
        } catch (InsufficientBalanceException e) {
            System.out.println(e.getMessage());
        } catch (IllegalArgumentException e) {
            System.out.println(e.getMessage());
        } finally {
            System.out.println("Bank transaction completed.");
        }
    }
}
```

OUTPUT:
Enter balance: 10000
Enter withdrawal amount: 3000
Withdrawal Successful
Remaining Balance: 7000.0
Bank transaction completed.

Q12. Hospital Patient Age Validation
Create a Java program for hospital patient registration.
Accept:
• Patient name
• Patient age
The valid age range is 0–120.

Create a custom exception named InvalidPatientAgeException.
If the age is outside the valid range, throw the custom exception.
Also handle the situation where the user enters non-numeric input for age.
Concepts: Custom exception, throw, catch, NumberFormatException

```
import java.util.Scanner;

class InvalidPatientAgeException extends Exception {
    InvalidPatientAgeException(String msg) {
        super(msg);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter patient name: ");
            String name = sc.nextLine();

            System.out.print("Enter patient age: ");
            int age = Integer.parseInt(sc.nextLine());

            if (age < 0 || age > 120)
                throw new InvalidPatientAgeException("Invalid patient age");

            System.out.println("Patient Registered: " + name);
            System.out.println("Age: " + age);

        } catch (NumberFormatException e) {
            System.out.println("Age must be numeric");
        } catch (InvalidPatientAgeException e) {
            System.out.println(e.getMessage());
        }
    }
}
```

OUTPUT:
Enter patient name: Sumit
Enter patient age: 25
Patient Registered: Sumit
Age: 25

Q13. Pharmaceutical Drug Dosage Validation
Create a Java program for validating a patient's drug dosage.
Accept:
• Patient name
• Drug name
• Dosage in mg
The valid dosage range is 1 mg–1000 mg.
Create a custom exception named InvalidDosageException.
If the dosage is less than or equal to 0, or greater than 1000, throw the custom exception. Also handle invalid numeric input and use a finally block to indicate that dosage validation is completed.
Concepts: Custom exception, throw, try-catch, finally, NumberFormatException

```
import java.util.Scanner;

class InvalidDosageException extends Exception {
    InvalidDosageException(String msg) {
        super(msg);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter patient name: ");
            String patient = sc.nextLine();

            System.out.print("Enter drug name: ");
            String drug = sc.nextLine();

            System.out.print("Enter dosage (mg): ");
            double dosage = Double.parseDouble(sc.nextLine());

            if (dosage <= 0 || dosage > 1000)
                throw new InvalidDosageException("Invalid Dosage");

            System.out.println("Patient: " + patient);
            System.out.println("Drug: " + drug);
            System.out.println("Dosage: " + dosage + " mg");

        } catch (NumberFormatException e) {
```

```
            System.out.println("Invalid numeric input");
        } catch (InvalidDosageException e) {
            System.out.println(e.getMessage());
        } finally {
            System.out.println("Dosage validation completed.");
        }
    }
}
```

OUTPUT:
Enter patient name: Sumit
Enter drug name: Paracetamol
Enter dosage (mg): 500
Patient: Sumit
Drug: Paracetamol
Dosage: 500.0 mg
Dosage validation completed.

Q14. Online Examination System
Create a Java program for an online examination system.
Accept a student's marks.
Apply the following rules:
0–100 → Valid
< 0 → Invalid
> 100 → Invalid
Create a custom exception named InvalidExamMarksException.
The program should:
1. Accept marks.
2. Validate the marks.
3. Throw the custom exception if marks are invalid.
4. Display PASS if marks are 40 or above.
5. Display FAIL if marks are below 40.
6. Handle invalid numeric input.
7. Use finally to display:
Exam evaluation completed.
Concepts: Custom exception, throw, catch, finally, NumberFormatException

```
import java.util.Scanner;

class InvalidExamMarksException extends Exception {
    InvalidExamMarksException(String msg) {
```

```
        super(msg);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter marks: ");
            int marks = Integer.parseInt(sc.nextLine());

            if (marks < 0 || marks > 100)
                throw new InvalidExamMarksException("Invalid Marks");

            if (marks >= 40)
                System.out.println("PASS");
            else
                System.out.println("FAIL");

        } catch (NumberFormatException e) {
            System.out.println("Invalid numeric input");
        } catch (InvalidExamMarksException e) {
            System.out.println(e.getMessage());
        } finally {
            System.out.println("Exam evaluation completed.");
        }
    }
}
```

OUTPUT:
Enter marks: 75
PASS
Exam evaluation completed.

Q15. Pharmacy Inventory System
Create a Java program for a pharmacy inventory system.
Accept:
• Medicine name
• Available quantity
• Required quantity
Apply the following rules:
1. Quantity cannot be negative.

2. Required quantity cannot be greater than available quantity.
3. If any quantity is negative, throw InvalidQuantityException.
4. If required quantity is greater than available quantity, throw InsufficientMedicineStockException.
5. Handle invalid numeric input.
6. Use multiple catch blocks.
7. Use a finally block to indicate that the inventory transaction is completed.
Concepts: Multiple custom exceptions, throw, multiple catch, finally, NumberFormatException

```
import java.util.Scanner;

class InvalidQuantityException extends Exception {
    InvalidQuantityException(String msg) {
        super(msg);
    }
}

class InsufficientMedicineStockException extends Exception {
    InsufficientMedicineStockException(String msg) {
        super(msg);
    }
}

public class Main {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);

        try {
            System.out.print("Enter medicine name: ");
            String medicine = sc.nextLine();

            System.out.print("Enter available quantity: ");
            int available = Integer.parseInt(sc.nextLine());

            System.out.print("Enter required quantity: ");
            int required = Integer.parseInt(sc.nextLine());

            if (available < 0 || required < 0)
                throw new InvalidQuantityException("Quantity cannot be negative");

            if (required > available)
                throw new InsufficientMedicineStockException(
                    "Insufficient Medicine Stock");
```

```
            System.out.println("Medicine: " + medicine);
            System.out.println("Transaction Successful");
            System.out.println("Remaining Stock: " + (available - required));

        } catch (NumberFormatException e) {
            System.out.println("Invalid numeric input");
        } catch (InvalidQuantityException e) {
            System.out.println(e.getMessage());
        } catch (InsufficientMedicineStockException e) {
            System.out.println(e.getMessage());
        } finally {
            System.out.println("Inventory transaction completed.");
        }
    }
}
```

OUTPUT:

```
Enter medicine name: Paracetamol
Enter available quantity: 100
Enter required quantity: 30
Medicine: Paracetamol
Transaction Successful
Remaining Stock: 70
Inventory transaction completed.
```